



Complete Project Explanation - From Zero to Hero

Welcome! Let's Learn Everything From Scratch

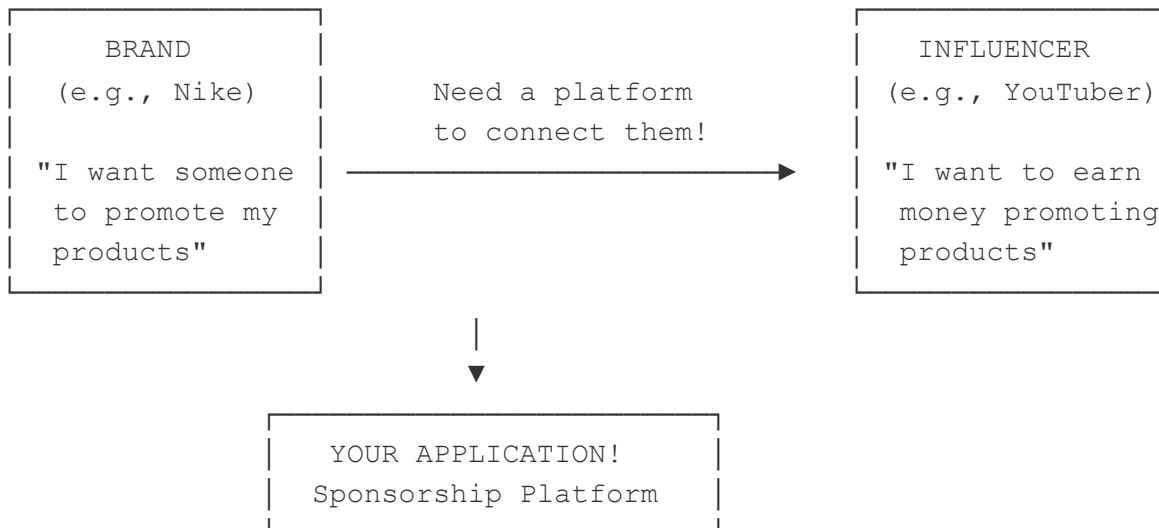
Don't worry if you have zero knowledge - I'll explain **everything** like you're learning for the first time.

PART 1: WHAT IS THIS PROJECT?



The Big Picture

Imagine Instagram influencers and companies wanting to work together:



Your app is like a matchmaking service between:

- **Brands** (companies like Nike, Samsung) who want marketing
 - **Influencers** (YouTubers, Instagrammers) who can promote products
-

Three Types of Users

1. ADMIN

- The boss of the platform
- Can see everything, delete users, view statistics
- Like the "super user" of the system

2. BRAND

- Companies that want marketing
- They CREATE campaigns (marketing projects)
- Example: "Nike wants 5 influencers to promote new shoes"

3. INFLUENCER

- Content creators with followers
- They APPLY to campaigns
- Example: "I can promote Nike shoes to my 100K followers"

What is a "Campaign"?

A campaign is a **marketing project**. Think of it like a job posting:

CAMPAIGN EXAMPLE
Name: "Summer Shoes Promotion 2026"
Brand: Nike
Platform: Instagram
Budget: \$5,000
Description: "Promote our new summer collection"
Requirements: "Must have 10K+ followers"
Duration: June 1 - June 30, 2026
Status: ACTIVE (accepting applications)



The Complete Business Flow

Here's how the app works step by step:

STEP 1: REGISTRATION

Brand signs up → "I'm Nike, here's my email/password"

Influencer signs up → "I'm John, I have 50K YouTube subscribers"

STEP 2: BRAND CREATES CAMPAIGN

Nike logs in → Creates "Summer Promotion" campaign

Sets budget: \$5,000

Sets platform: Instagram

Sets requirements: "10K+ followers"

STEP 3: INFLUENCER BROWSES & APPLIES

John logs in → Sees Nike's campaign

→ Clicks "Apply"

→ Writes proposal: "I can make 3 posts for you!"

→ Status: PENDING (waiting for Nike to respond)

STEP 4: BRAND REVIEWS APPLICATIONS

Nike logs in → Sees John's application

→ Reads his proposal

→ Either ACCEPTS ✓ or REJECTS ✗

STEP 5: WORK HAPPENS (Outside the app)

If accepted, John creates Instagram posts for Nike

(This happens in real life, not in the app)

STEP 6: PAYMENT

Nike creates payment → \$1,000 to John

Payment status: PENDING → COMPLETED

John receives money! 💰

STEP 7: RATINGS

Nike rates John: ★★★★★ "Great work!"

John rates Nike: ★★★★★ "Good communication"

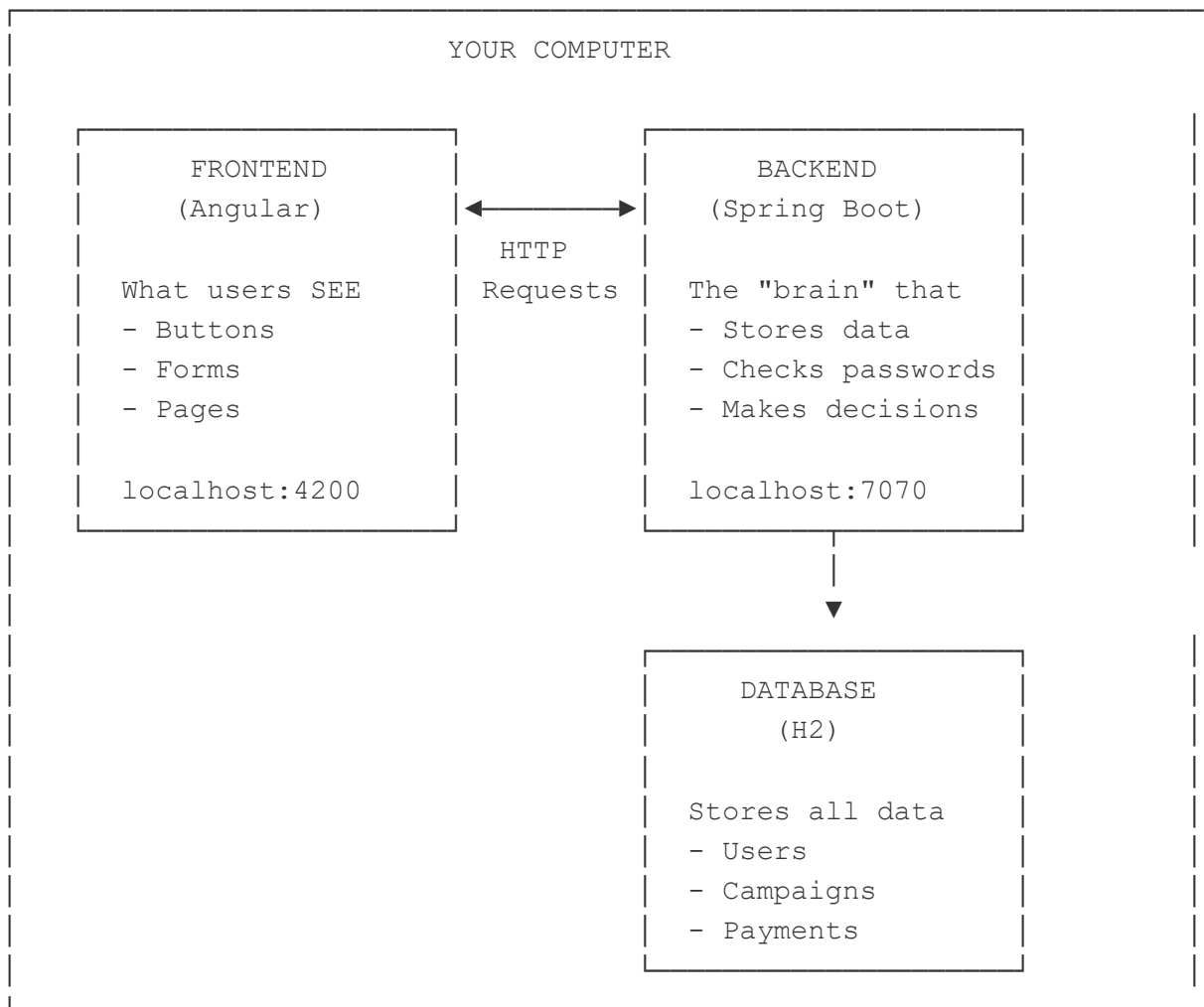
PART 2: TECHNOLOGY EXPLAINED

(For Beginners)



The Two Parts of Your Application

Your app has TWO separate parts that talk to each other:



FRONTEND (Angular) - The Face of Your App

What is Angular?

Angular is a **framework** (a set of pre-written code) that helps you build websites.

Think of it like this:

- Without Angular: You write 10,000 lines of code
- With Angular: You write 2,000 lines and Angular handles the rest

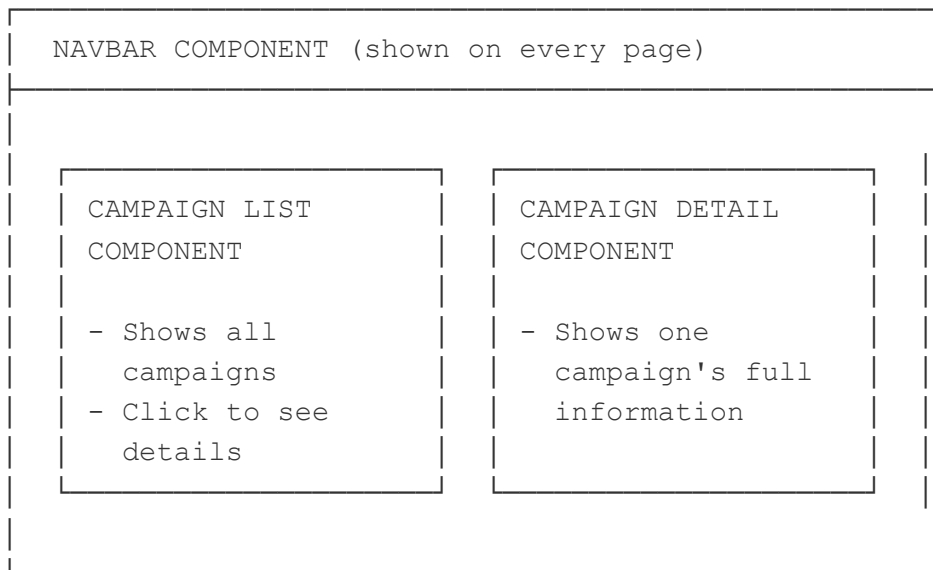
What does the Frontend do?

1. Shows **login/signup forms**
2. Displays **campaigns** in a nice list
3. Shows **dashboards** for each user type
4. Sends user actions to the backend

Key Frontend Concepts:

1. Components - Building blocks of pages

Your app is made of small pieces called "components":



2. Services - Code that talks to the backend

```
// This is a "service" - it sends requests to the backend
// Example: When you click "Login" button

authService.login(email, password)
  ↓
Sends to: http://localhost:7070/api/auth/login
  ↓
Backend checks if password is correct
  ↓
Returns: "Yes, here's your login token" or "No, wrong password"
```

3. Forms - Where users type information

SIGNUP FORM	
Username: [_____]	← Must be unique
Email: [_____]	← Must be valid email
Password: [_____]	← Must have letters, numbers, and @
Role: [Brand ▼]	
[Create Account]	

BACKEND (Spring Boot) - The Brain of Your App

What is Spring Boot?

Spring Boot is a **Java framework** that helps you build the "server" - the part that:

- Stores data in a database
- Checks if passwords are correct
- Decides who can access what

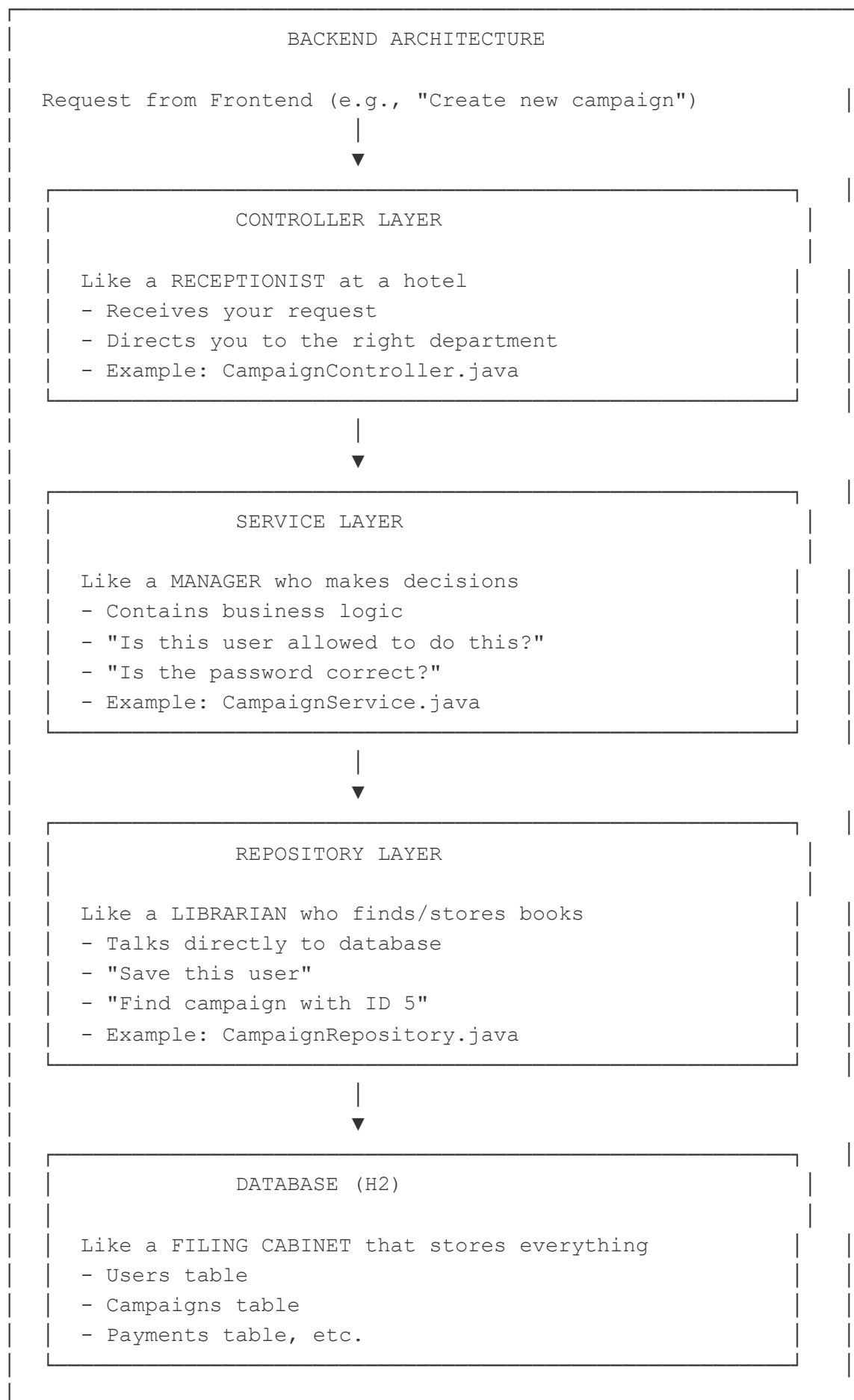
What does the Backend do?

1. **Receives requests** from the frontend

2. **Processes them** (check password, save data, etc.)

3. **Sends responses** back to the frontend

How it's organized (Layer by Layer):





DATABASE - Where Data Lives

What is H2?

H2 is a **database** - think of it as a smart Excel spreadsheet that stores all your data.

Why H2?

- It's **in-memory** (lives in your computer's RAM)
- **No installation needed** - just works
- **Resets when you restart** the app (good for testing)
- For real apps, you'd use MySQL or PostgreSQL

Your Database Tables:

DATABASE TABLES

USERS TABLE (like a spreadsheet of all users)

ID	Name	Email	Role
1	Admin	admin@sponsor.com	ADMIN
2	Nike	nike@example.com	BRAND
3	John	john@gmail.com	INFLUENCER

CAMPAIGNS TABLE

ID	Name	Budget	Brand_ID
1	Summer Promo	5000	2 (Nike)
2	Winter Sale	3000	2 (Nike)

└─ Brand_ID = 2 means this campaign
 belongs to Nike (User ID 2)

PART 3: HOW THINGS CONNECT (The Magic)



What Happens When You Click "Login"?

Let's trace exactly what happens:

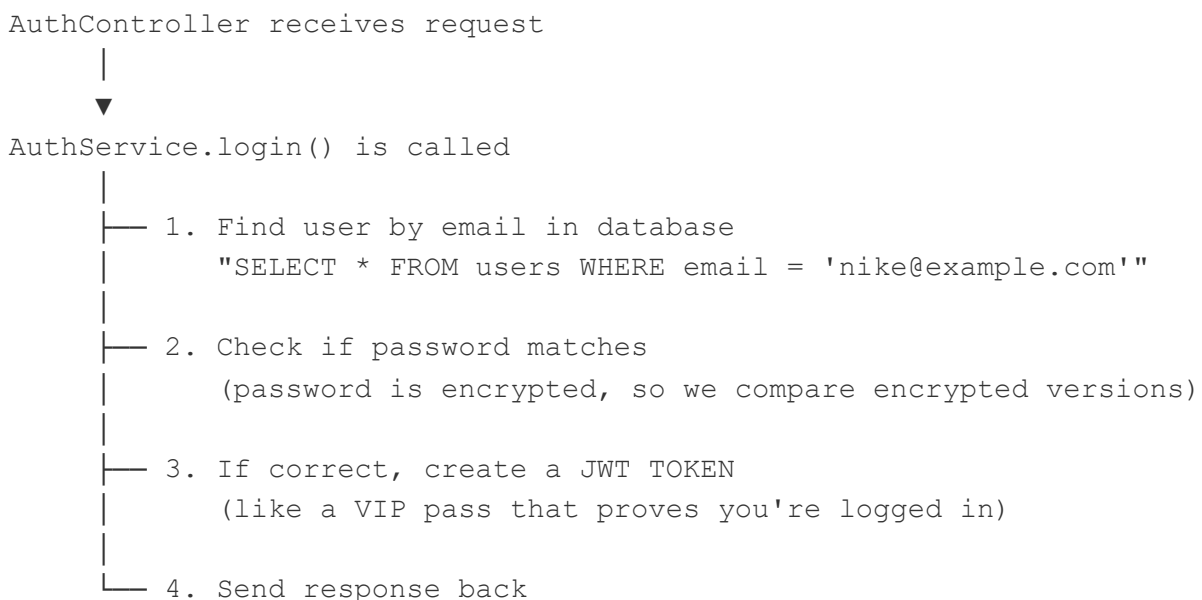
STEP 1: You type email & password, click "Login"

[nike@example.com]	
[●●●●●●●●]	
[LOGIN]	← You click this

STEP 2: Angular sends HTTP request to backend



STEP 3: Backend processes the login



STEP 4: Backend sends response



```
|      "id": 2,  
|      "name": "Nike",  
|      "email": "nike@example.com",  
|      "role": "BRAND"  
|    }  
|←
```

STEP 5: Angular stores the token & redirects

- Token is saved in browser's localStorage
- User info is saved
- Page redirects to Brand Dashboard
- "Welcome, Nike!" appears on screen



What is JWT Token? (Very Important!)

The Problem:

HTTP is "stateless" - the server forgets you after each request. It's like a receptionist with amnesia - they forget you every time.

The Solution: JWT (JSON Web Token)

It's like a **VIP wristband** at a concert:

WITHOUT JWT:

Request 1: "I'm Nike, show me
my campaigns"
Server: "Who are you? Login
first!"

Request 2: "I'm Nike, I just
logged in!"
Server: "I don't remember you.
Login again!"

WITH JWT:

Request 1: "Here's my token"
Server: "Ah yes, you're Nike.
Here's your data"

Request 2: "Here's my token"
Server: "Welcome back Nike,
here's more data"

The token PROVES who you are!

What's inside a JWT?

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJuaWtlQGV4YW1wbGUuY29tIiwiaWF0IjoxNjE...

This looks like gibberish, but it contains:

DECODED JWT TOKEN:

```
{
  "sub": "nike@example.com", ← User email
  "iat": 1715673600,          ← Issued at
  "exp": 1715760000           ← Expires at
}
```

+ A secret signature that can't be faked

Password Security (How We Protect Passwords)

The Problem:

If someone hacks your database, they can see all passwords!

The Solution: BCrypt Encryption (One-Way Hashing)

WHAT HAPPENS WHEN USER REGISTERS:

User types: "MyPassword123@"



BCrypt Encryption



Stored in DB: "\$2a\$10\$N9qo8uLOickgx2ZMRZoMy..."

This CANNOT be reversed! Even we don't know the original password.

WHAT HAPPENS WHEN USER LOGS IN:

User types: "MyPassword123@"



BCrypt Encryption (same algorithm)



"\$2a\$10\$N9qo8uLOickgx2ZMRZoMy..."



COMPARE with database value



Same? → LOGIN SUCCESS!

Different? → LOGIN FAILED!

PART 4: CODE EXPLAINED (Line by



Understanding the Folder Structure

```
sponsorship-app-backend/
├── src/main/java/com/myapp/sponsorshipapp/
│   ├── config/                ← Settings (security, CORS)
│   │   ├── SecurityConfig.java    "Who can access what?"
│   │   └── DataInitializer.java    "Create demo users on startup"
│   ├── controller/            ← Receives HTTP requests
│   │   ├── AuthController.java    "Handle login/signup"
│   │   ├── CampaignController.java "Handle campaign operations"
│   │   └── ...
│   ├── service/               ← Business logic (the brain)
│   │   ├── AuthService.java       "Check passwords, create tokens"
│   │   ├── CampaignService.java   "Create/update campaigns"
│   │   └── ...
│   ├── repository/           ← Database access
│   │   ├── UserRepository.java    "Find/save users"
│   │   ├── CampaignRepository.java "Find/save campaigns"
│   │   └── ...
│   ├── entity/               ← Database table definitions
│   │   ├── User.java             "What a user looks like"
│   │   ├── Campaign.java         "What a campaign looks like"
│   │   └── ...
│   ├── dto/                  ← Data shapes for requests/responses
│   │   ├── LoginRequest.java      "email + password"
│   │   ├── AuthResponse.java      "token + user info"
│   │   └── ...
│   ├── security/             ← JWT handling
│   │   ├── JwtTokenProvider.java  "Create and validate tokens"
│   │   └── JwtAuthFilter.java     "Check token on every request"
│   └── exception/            ← Error handling
│       └── GlobalExceptionHandler.java
└── src/main/resources/
    └── application.properties ← Configuration settings
```

```
└─ frontend/                                ← Angular application
  └─ src/app/
    ├── components/                         ← UI components (pages)
    ├── services/                           ← API calls
    ├── guards/                             ← Route protection
    └─ models/                             ← TypeScript interfaces
```



Key Code Examples Explained

1. Entity (Database Table Definition)

```
// User.java - This defines what a "user" looks like in the database

@Entity                                // "This class = a database table"
@Table(name = "users")                 // "Name the table 'users'"
public class User {

    @Id                                // "This is the primary key (unique identifier)"
    @GeneratedValue(strategy = GenerationType.IDENTITY) // "Auto-increment"
    private Long id;                    // User #1, #2, #3, etc.

    @Column(unique = true) // "No two users can have the same name"
    private String name;

    @Column(unique = true) // "No two users can have the same email"
    private String email;

    private String password; // Stored encrypted!

    @Enumerated(EnumType.STRING) // Store as "BRAND" not as number
    private Role role;          // ADMIN, BRAND, or INFLUENCER
}
```

2. Repository (Database Access)

```
// UserRepository.java - How we talk to the database

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring automatically creates the SQL for us!

    Optional<User> findByEmail(String email);
    // SQL: SELECT * FROM users WHERE email = ?

    boolean existsByEmail(String email);
    // SQL: SELECT COUNT(*) > 0 FROM users WHERE email = ?

    boolean existsByNameIgnoreCase(String name);
    // SQL: SELECT COUNT(*) > 0 FROM users WHERE LOWER(name) = LOWER(?)
}
```

3. Service (Business Logic)

```
// AuthService.java - The "brain" that makes decisions

@Service
public class AuthService {

    public AuthResponse register(RegisterRequest request) {

        // Step 1: Clean up the input
        String name = request.getName().trim();
        String email = request.getEmail().trim().toLowerCase();

        // Step 2: Check if username already exists
        if (userRepository.existsByNameIgnoreCase(name)) {
            throw new RuntimeException("Username already exists");
            // This stops everything and sends error to frontend
        }

        // Step 3: Check if email already exists
        if (userRepository.existsByEmail(email)) {
            throw new RuntimeException("Email already registered");
        }

        // Step 4: Create new user
        User user = new User();
        user.setName(name);
        user.setEmail(email);
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        //          ↑ IMPORTANT: Password is encrypted here!
        user.setRole(Role.valueOf(request.getRole()));

        // Step 5: Save to database
        userRepository.save(user);

        // Step 6: Create JWT token so user is logged in immediately
        String token = tokenProvider.generateToken(authentication);

        // Step 7: Return response
        return new AuthResponse(token, user.getId(), user.getName(), ...);
    }
}
```

4. Controller (Receives HTTP Requests)

```
// AuthController.java - The "receptionist"

@RestController          // "This class handles HTTP requests"
@RequestMapping("/api/auth") // "All URLs start with /api/auth"
public class AuthController {

    @PostMapping("/register") // Handles POST to /api/auth/register
    public ResponseEntity<?> register(
        @Valid @RequestBody RegisterRequest request) {
        // ↑ @Valid = check validation rules (email format, etc.)
        //      ↑ @RequestBody = get data from request body (JSON)

        try {
            AuthResponse response = authService.register(request);
            return ResponseEntity.ok(response); // 200 OK + data
        } catch (Exception e) {
            return ResponseEntity.badRequest() // 400 Bad Request
                .body(new ApiResponse(false, e.getMessage()));
        }
    }

    @PostMapping("/login") // Handles POST to /api/auth/login
    public ResponseEntity<?> login(@Valid @RequestBody LoginRequest request) {
        // Similar to register...
    }
}
```



Frontend Code Explained

1. Service (Talks to Backend)

```
// auth.service.ts - Communicates with backend

@Injectable({ providedIn: 'root' })
export class AuthService {

  private apiUrl = 'http://localhost:7070/api/auth';

  // Send login request to backend
  login(request: LoginRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${this.apiUrl}/login`, request)
      .pipe(
        tap(response => {
          // Save token in browser storage
          localStorage.setItem('token', response.token);
          // Save user info
          localStorage.setItem('user', JSON.stringify(response));
        })
      );
  }

  // Check if user is logged in
  isLoggedIn(): boolean {
    return !!localStorage.getItem('token'); // true if token exists
  }

  // Get current user's role
  getCurrentUser(): User {
    return JSON.parse(localStorage.getItem('user'));
  }
}
```

2. Component (UI + Logic)

```
// login.component.ts - The login page

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {

  // Form definition
  loginForm: FormGroup;

  constructor(
    private fb: FormBuilder,
    private authService: AuthService,
    private router: Router
  ) {
    // Create form with validation rules
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  // When user clicks "Login" button
  onSubmit(): void {
    if (this.loginForm.invalid) {
      return; // Don't submit if form is invalid
    }

    this.authService.login(this.loginForm.value).subscribe({
      next: () => {
        // Success! Go to dashboard
        this.router.navigate(['/dashboard']);
      },
      error: (err) => {
        // Show error message
        alert('Login failed: ' + err.error.message);
      }
    });
  }
}
```


3. Template (HTML)

```
<!-- login.component.html - What user sees -->

<div class="login-container">
  <mat-card class="login-card">
    <mat-card-title>Welcome Back</mat-card-title>

    <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">

      <!-- Email field -->
      <mat-form-field>
        <input matInput formControlName="email"
          placeholder="Enter your email">

        <!-- Show error if invalid -->
        <mat-error *ngIf="loginForm.get('email')?.hasError('required')">
          Email is required
        </mat-error>
        <mat-error *ngIf="loginForm.get('email')?.hasError('email')">
          Invalid email format
        </mat-error>
      </mat-form-field>

      <!-- Password field -->
      <mat-form-field>
        <input matInput type="password" formControlName="password"
          placeholder="Enter your password">
      </mat-form-field>

      <!-- Submit button -->
      <button mat-raised-button color="primary" type="submit"
        [disabled]="loginForm.invalid">
        Sign In
      </button>

    </form>
  </mat-card>
</div>
```

PART 5: CONFIGURATION FILES

EXPLAINED

application.properties (Backend Settings)

```
# Application name
spring.application.name=sponsorship-app-backend

# Server runs on port 7070 (not default 8080)
server.port=7070

# Database settings (H2 in-memory database)
spring.datasource.url=jdbc:h2:mem:sponsorshipdb
#           ↑ "jdbc:h2:mem:" means in-memory
#           ↑ "sponsorshipdb" is database name

spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa           # Default H2 username
spring.datasource.password=            # No password

# H2 Console (web interface to see database)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
# Visit: http://localhost:7070/h2-console

# JPA settings
spring.jpa.hibernate.ddl-auto=create-drop
#           ↑ "create-drop" means:
#           - CREATE tables when app starts
#           - DROP (delete) tables when app stops
#           Good for development, BAD for production!

spring.jpa.show-sql=true  # Show SQL queries in console (for debugging)

# JWT Settings
app.jwt.secret=mySecretKey123...  # Secret key to sign tokens
app.jwt.expiration=86400000        # Token valid for 24 hours (in milliseconds)

# CORS - Allow frontend to talk to backend
app.cors.allowed-origins=http://localhost:4200
#           ↑ Frontend URL
```

PART 6: VALIDATION RULES (What We Check)

Username Validation

- ✓ Required (can't be empty)
- ✓ Minimum 3 characters
- ✓ Must be unique (no duplicates)
- ✓ Case-insensitive check ("John" = "john" = "JOHN")

Email Validation

- ✓ Required
- ✓ Must be valid format (something@something.com)
- ✓ Must be unique
- ✓ Stored in lowercase

Password Validation

- ✓ Required
- ✓ Minimum 6 characters
- ✓ Must contain at least one letter (a-z or A-Z)
- ✓ Must contain at least one number (0-9)
- ✓ Must contain @ symbol

Valid examples: abc123@, Pass1@word, test@99

Invalid examples: password, 123456, abc@def (no number)

PART 7: QUICK REFERENCE CARD

URLs to Remember

URL	What it does
http://localhost:4200	Frontend (Angular)
http://localhost:7070	Backend (Spring Boot)
http://localhost:7070/h2-console	Database viewer

API Endpoints

Method	URL	What it does
POST	/api/auth/register	Create new user
POST	/api/auth/login	Login
GET	/api/campaigns	Get all campaigns
POST	/api/campaigns	Create campaign
POST	/api/sponsorship/apply	Apply to campaign
POST	/api/payments	Create payment
POST	/api/ratings	Add rating

Demo Accounts (Pre-created)

Role	Email	Password
Admin	admin@sponsorship.com	admin123
Brand	brand@example.com	brand123
Influencer	influencer@example.com	influencer123



INTERVIEW CHEAT SHEET

If asked "Explain your project in 30 seconds":

"I built a sponsorship platform using Spring Boot and Angular. Brands create marketing campaigns, Influencers apply to them, and the platform manages payments and ratings. I implemented JWT authentication for security, used Spring Data JPA for database operations, and Angular Material for the UI."

If asked "What was the most challenging part?":

"Implementing the security layer with JWT. I had to understand how tokens work, create filters to validate them on every request, and handle token expiration properly."

If asked "What would you improve?":

1. Add real payment integration (Stripe/PayPal)
2. Implement email notifications
3. Add file upload for profile pictures
4. Switch to PostgreSQL for production
5. Add unit and integration tests

You've got this! 💪

Remember: It's okay to say "I'm still learning" in an interview. What matters is showing you understand the concepts and can explain your thinking.